Complete finite-example program for rank contraction

Human Compression project

Private appendix; independent review pending

This is the complete mathematical GAP program used for the displayed four-dimensional matrix example and the three literal diagrams for A_{11}. It is a finite certificate for those examples, not a premise of the uniform contraction theorem or its infinite families. GAP group computation is trusted. The operational replay driver and scoped Lean module are separate files.

```
# Complete finite source examples; these are not premises of the uniform theorem.
SizeScreen([240,60]);;
Need:=function(b,m)if not b then Error(m);fi;end;;
Par:=function(xs,I)return Group(Concatenation([One(xs[1])],xs{I}));end;;
CheckC:=function(xs)local sets,gs,j,k;
 Need(Length(Set(xs))=Length(xs) and ForAll(xs,x->Order(x)=2),"marks");
 for j in [1..Length(xs)] do for k in [1..Length(xs)] do
  if AbsInt(j-k)>1 then Need(xs[j]*xs[k]=xs[k]*xs[j],"string");fi;
 od;od;
 sets:=Combinations([1..Length(xs)]);gs:=List(sets,I->Par(xs,I));
 for j in [1..Length(sets)] do for k in [j..Length(sets)] do
  Need(Intersection(gs[j],gs[k])=Par(xs,Intersection(sets[j],sets[k])),"intersection");
 od;od;return Length(sets)*(Length(sets)+1)/2;end;;
TypeOf:=function(xs)return List([1..Length(xs)-1],j->Order(xs[j]*xs[j+1]));end;;
f:=GF(3);z:=One(f);;
F:=z*[[1,1,0,0],[1,2,1,0],[0,1,1,2],[0,0,2,1]];;
xs:=z*[
 [[2,0,0,0],[1,1,0,0],[0,0,1,0],[0,0,0,1]],
 [[2,1,0,0],[0,1,0,0],[0,1,2,0],[0,0,0,2]],
 [[1,0,0,0],[0,1,1,0],[0,0,2,0],[0,0,2,1]],
 [[2,0,0,0],[0,2,0,0],[0,0,2,1],[0,0,0,1]]];;
Need(DeterminantMat(F)<>Zero(f),"form nonsingular");
Need(ForAll(xs,x->x*F*TransposedMat(x)=F),"form preservation");
zero_vectors:=Number(Tuples(Elements(f),4),v->v*F*v=Zero(f));
Need(zero_vectors=21,"minus-type singular count");
G:=Group(xs);ys:=[xs[2],xs[1]*xs[3],xs[4]];;
Need(Size(G)=1440 and Group(ys)=G,"full matrix group");
Need(TypeOf(xs)=[4,4,6] and TypeOf(ys)=[6,6],"source types");
checks:=CheckC(xs)+CheckC(ys);Need(not xs[1] in Group(ys{[2,3]}),"recovery hypothesis fails");
Print("MATRIX order=",Size(G)," zero_vectors=",zero_vectors," unordered_IP=",checks," recovery=false\n");

# Exact Figure1 path labels, vertices numbered from left to right.
figures:=[[[0,2],[1],[0],[1],[2],[3],[4],[5],[4],[3,5]],
 [[0],[1],[0,2],[1],[2],[3],[4],[5],[4],[3,5]],
 [[0],[1],[0,2],[1],[2],[3],[4],[3,5],[4],[5]]];;
types:=[[5,3,6,3,5],[5,5,6,3,5],[5,5,6,5,5]];;
allchecks:=0;;
for i in [1..3] do
 xs:=List([0..5],j->());
 for e in [1..10] do for lab in figures[i][e] do xs[lab+1]:=xs[lab+1]*(e,e+1);od;od;
 Need(TypeOf(xs)=types[i] and Group(xs)=AlternatingGroup(11),"literal A11 graph");
 allchecks:=allchecks+CheckC(xs);
 ys:=Concatenation([xs[2],xs[1]*xs[3]],xs{[4..6]});
 rev:=Reversed(xs);ds:=Concatenation([rev[2],rev[1]*rev[3]],rev{[4..6]});
 lo:=List(Orbits(Group(ys),[1..11]),Length);ro:=List(Orbits(Group(ds),[1..11]),Length);
 Need(Length(lo)>1 and Length(ro)>1,"both reductions are proper by intransitivity");
 Print("A11 row=",i," type=",types[i]," left_orbits=",lo," right_orbits=",ro,"\n");
od;
Print("PASS matrix_unordered_IP=",checks," A11_unordered_IP=",allchecks,"\n");
QUIT_GAP(0);
```